자기소개서 원본

> 궁금한 것을 파고들어 의미를 발견하고, 실제로 동작하는 결과로 만들며, 사람들과 함께 앞으로 나아가는 사람.

이 문서는 여러 경험을 보존하는 공통 자기소개서 원본입니다. 실제 지원 시에는 회사와 직무, 자기소개서 문항에 맞춰 필요한 경험을 골라 분량과 강조점을 조정합니다.

1. 자기소개

저는 무언가를 배울 때 사용법만 외우기보다 왜 그렇게 동작하는지 이해하고 직접 실행해 보는 과정을 중요하게 생각합니다. 오류를 해결하거나 복잡하게 얽힌 흐름이 정리될 때 특히 재미를 느낍니다.

인제대학교에서 반도체공학을 전공하며 공정 실습과 클린룸 실습을 경험했습니다. 대학 프로젝트에서 Arduino와 Bluetooth, MIT App Inventor를 활용해 센서 정보를 휴대폰으로 전달하는 화재 알림 시제품을 만들면서 소프트웨어가 실제 장치와 연결되는 과정에 관심을 갖게 됐습니다. 이후 ITWill 백엔드 개발 교육에서 Java와 JavaScript를 중심으로 웹 개발을 배우고 팀 프로젝트를 진행했습니다. 최근에는 SKT K-뉴딜 ALEPH 과정에서 Python, Linux, 데이터 분석, AI 활용과 인프라 보안 실습을 경험하고 있습니다.

개인 프로젝트인 Maple Growth Tracker에서는 NEXON Open API 데이터를 가져와 가공하고, 성장 기록을 저장·분석하는 웹서비스를 만들어 Render에 배포했습니다. ALEPH 캡스톤 SENTRA에서는 직원 가입과 계정 상태 관리, 접근 정책, 기간제 권한 요청·승인·회수 기능을 Flask API와 SQLite를 중심으로 구현하고 주요 시나리오를 검증했습니다.

저는 아직 모든 기술을 능숙하게 다룬다고 말하기보다, 모르는 부분을 구체적인 질문으로 나누고 코드와 실행 결과를 비교하며 이해의 범위를 넓혀 가는 사람이라고 소개하고 싶습니다.

2. 성장 과정: 전공과 소프트웨어를 연결하다

처음부터 소프트웨어 개발을 진로로 정한 것은 아닙니다. 대학에서 나노융합·반도체공학을 공부하며 제조 공정과 실험을 경험했고, Arduino 기반 프로젝트를 진행하면서 소프트웨어가 실제 장치의 동작과 연결되는 과정에 흥미를 느꼈습니다. 이 관심을 더 깊게 알아보기 위해 ITWill 백엔드 개발 교육을 수강하며 Java, JavaScript, Spring 계열 기술과 데이터베이스를 학습했습니다.

ITWill 팀 프로젝트에서는 Stock-Mate, MrBean, Packtory_DE를 진행했습니다. 재고 관리 시스템의 Category CRUD와 Stock 기능, 생산관리 시스템의 원자재·완제품·BOM·입고 및 LOT 기능, MES형 프로젝트의 직원·부서·직급·제품·창고·품질·근무관리 기능을 다뤘습니다. 이 과정에서 제조 업무가 단순히 화면이나 데이터 하나로 이루어지는 것이 아니라 여러 업무와 데이터가 서로 연결된다는 점을 배웠습니다.

현재는 반도체 전공과 소프트웨어 경험을 서로 분리된 이력으로 두기보다, 제조 현장의 데이터와 업무 흐름을 IT 시스템으로 연결하는 방향에도 관심을 두고 있습니다. MES·ERP, 제조 IT와 백엔드 개발을 함께 살펴보며 제가 가진 경험을 활용할 수 있는 직무를 구체화하고 있습니다.

3. 문제 해결 방식: 화면부터 데이터까지 흐름을 따라가기

개발 중 문제가 생기면 우선 문제가 발생하는 상황을 재현하고, 요청과 응답 및 데이터가 전달되는 흐름을 살펴봅니다. 수정한 뒤에는 다시 실행해 같은 문제가 해결됐는지 확인하려고 합니다.

MrBean 프로젝트에서는 원자재와 완제품 CRUD, BOM, 원자재 입고와 LOT 연결 기능을 작업했습니다. 화면에서 입력한 값이 JSP에서 Controller, Service, DAO, MyBatis Mapper를 거쳐 DB에 저장되고 다시 화면에 표시되는 구조를 여러 기능에서 다뤘습니다. 처음에는 한 기능을 수정할 때 어느 파일을 확인해야 할지 찾는 데 시간이 걸렸지만, 요청이 지나가는 계층을 하나씩 따라가며 원인을 좁히는 방법을 익혔습니다.

Stock-Mate에서는 Category CRUD와 Stock 기능을 중심으로 Controller, Service, DAO, Mapper, VO와 JSP를 함께 수정했습니다. 카테고리는 상품과 재고 등 다른 기능에서도 사용하는 데이터였기 때문에, 한 화면만 정상적으로 동작하는지 보는 것이 아니라 다른 기능에서 해당 데이터를 어떻게 사용하는지도 살펴야 했습니다.

이런 경험을 통해 기능 하나의 동작만 확인하는 것보다 화면, 서버 로직, SQL과 데이터베이스가 연결되는 전체 흐름을 이해하는 것이 중요하다는 점을 배웠습니다.

4. 데이터에 근거해 판단한 경험

ALEPH T10 타이핑 연구에서는 시각적 효과가 타이핑 성능에 영향을 주는지 궁금해 직접 실험을 구성했습니다. JavaScript로 시각적 효과 ON/OFF 조건의 타이핑 페이지를 만들고 총 10회 실험을 진행한 뒤, 결과를 CSV로 저장해 Python으로 분석했습니다.

처음에는 시각적 효과가 있을 때 타이핑 속도와 정확도가 낮아질 것이라고 예상했습니다. 분석 결과에서도 ON 조건의 평균 타이핑 속도는 566.00으로 OFF 조건의 594.67보다 낮았고, 정확도 역시 ON 99.47, OFF 100으로 나타났습니다.

저는 결과를 가설에 맞춰 해석하기보다 실제 데이터를 기준으로 비교했습니다. 다만 실험 횟수와 표본이 제한적이므로 이번 실험에서 관찰한 결과로 해석했습니다. 이 경험을 통해 분석은 처음 생각한 답을 증명하는 과정이 아니라, 데이터를 정확히 살펴보고 결과를 있는 그대로 설명하는 과정이라는 점을 배웠습니다.

5. 실제 사용할 수 있는 결과물을 만든 경험

Maple Growth Tracker는 메이플스토리 캐릭터의 성장 정보를 편하게 확인하고 싶어서 시작한 개인 프로젝트입니다. NEXON Open API에서 캐릭터 데이터를 가져와 화면에 표시하고, 날짜별 성장 기록을 저장해 성장량과 랭킹 변화를 비교할 수 있도록 만들었습니다.

최근 기록만으로는 장기간의 성장 흐름을 확인하기 어려워 30일·90일 기간 조회 기능을 확장했습니다. 누락된 날짜를 구간별로 나누어 요청하고 기존 기록을 재사용하도록 했으며, API에서 제공하지 않는 날짜와 요청 오류를 구분했습니다. 성장 지표는 코드에서 계산하고 Chart.js로 시각화했으며, Gemini API와 OpenRouter를 연결해 계산된 결과를 자연어로 설명하는 기능도 추가했습니다.

개발을 마친 뒤에는 Render에 배포하고 실제 환경에서 데이터베이스 연결과 저장 상태를 확인했습니다. 로컬에서 애플리케이션이 실행되는 것과 배포 환경에서 데이터가 안정적으로 보존되는 것은 별개의 문제라는 점을 경험했습니다. 이 프로젝트를 통해 개인적인 필요에서 출발한 아이디어도 기능을 나누어 구현하면 실제 사용할 수 있는 웹서비스로 만들 수 있다는 자신감을 얻었습니다.

6. 협업 경험과 태도

팀 프로젝트를 진행하면서 각자 익숙한 기술과 구현 방법, 작업 순서가 다를 수 있다는 점을 배웠습니다. 내 방식만 고집하기보다 기존 프로젝트의 구조를 먼저 이해하고, 다른 팀원의 의견을 듣고, 기능이 서로 연결되는 지점을 조정하는 것이 중요하다고 생각합니다.

Packtory_DE에서는 기존에 만들어진 Spring Boot 구조에 적응하며 직원·부서·직급, 제품, 원자재, 창고, 입출고, 품질과 근무관리 기능을 작업했습니다. 처음 접한 공통코드와 상·하위 부서 코드 구조가 낯설었지만, 기존 데이터 흐름을 살펴보고 프로젝트에서 사용하던 방식에 맞춰 기능을 수정하려고 노력했습니다.

SENTRA에서는 프런트엔드와 백엔드를 나누어 개발하더라도 API 응답 형식과 데이터 구조가 맞아야 기능이 연결된다는 점을 경험했습니다. 또한 애플리케이션의 접근 정책과 네트워크 계층의 통제는 서로 다른 역할이므로, 실제 구현한 기능과 별도 환경에서 검증해야 하는 기능을 구분해 팀의 결과물을 설명하려고 했습니다.

7. 강점과 개선 중인 점

저의 강점은 궁금한 점을 그냥 넘기지 않고 직접 확인하려는 태도입니다. 오류가 발생하면 코드와 실행 결과를 살펴보고, 예상과 실제 결과가 다르면 원인을 다시 찾아보려고 합니다. Maple Growth Tracker에서는 API 데이터의 누락과 오류 응답을 구분했고, SENTRA에서는 권한 요청이 승인되기 전과 회수된 뒤에 접근 정책이 어떻게 적용되는지 확인했습니다.

반면 작업에 착수하면 눈앞의 문제 해결에 집중해 전체 요구사항이나 제출 조건을 뒤늦게 확인하는 경우가 있었습니다. 이를 개선하기 위해 작업 전에 요구사항과 현재 상태를 먼저 살펴보고, 일을 작은 단위로 나누어 진행하고 있습니다. 작업을 마친 뒤에는 실행 결과와 제출 조건을 다시 점검하는 습관도 기르고 있습니다.

새로운 기술을 사용할 때도 동작하는 코드를 만드는 데서 멈추지 않고, 왜 그렇게 설계했는지 다른 사람에게 설명할 수 있을 때까지 이해하려고 노력하고 있습니다.

8. 앞으로의 방향

저는 소프트웨어 개발 경험을 계속 쌓으면서 반도체·제조 전공과 연결되는 업무도 함께 탐색하고 있습니다. 백엔드·웹 개발, MES·ERP 관련 개발, 제조 IT 및 IT 시스템 운영 분야의 실제 업무 내용을 비교하며 제 경험과 잘 맞는 방향을 좁혀 가고자 합니다.

입사 후에는 모르는 업무를 추측으로 처리하기보다 필요한 자료를 찾아 배우고, 동료와 소통하며, 작업 결과를 직접 확인하는 방식으로 적응하겠습니다. 장기적으로는 반도체 전공에서 얻은 제조 환경에 대한 이해와 소프트웨어 프로젝트 경험을 연결해 실제 업무의 문제를 해결하는 사람이 되고 싶습니다.

자기소개 핵심 문장

> 궁금한 것을 파고들어 의미를 발견하고, 실제로 동작하는 결과로 만들며, 사람들과 함께 앞으로 나아가는 사람.

자기소개서 작성 원칙

- 회사와 직무, 문항의 의도에 맞게 경험을 선별한다.
- 경험하지 않은 내용이나 확인되지 않은 성과를 추가하지 않는다.
- 팀 프로젝트의 전체 기능과 본인의 직접 기여를 구분한다.
- AI가 도운 작업과 직접 수행·검증한 작업을 구분한다.
- 기술 이름을 나열하기보다 문제, 행동, 결과와 배운 점을 중심으로 쓴다.
- 최종 제출 전에 문장과 실제 경험이 일치하는지 확인한다.